第 7 章 RKNN C API 详细说明

7.1 API 硬件平台支持说明

RKNN C API 不同硬件平台支持如下：

	RKNN C API	RK356X	RK3588	RV1106/RV1103
1	rknn_init	√	√	√
2	rknn_set_core_mask	×	√	×
3	rknn_dup_context	√	√	×
4	rknn_destroy	√	√	√
5	rknn_query	√	√	√
6	rknn_inputs_set	√	√	×
7	rknn_run	√	√	√
8	rknn_wait	×	×	×
9	rknn_outputs_get	√	√	×
10	rknn_outputs_release	√	√	√
11	rknn_create_mem_from_mb_blk	×	×	×
12	rknn_create_mem_from_phys	√	√	√
13	rknn_create_mem_from_fd	√	√	√
14	rknn_create_mem	√	√	√
15	rknn_destroy_mem	√	√	√
16	rknn_set_weight_mem	√	√	√
17	rknn_set_internal_mem	√	√	√
18	rknn_set_io_mem	√	√	√

rknn_query 函数不同平台支持的查询参数如下：

	rknn_query params	RK356X	RK3588	RV1106/RV1103
1	RKNN_QUERY_IN_OUT_NUM	√	√	√
2	RKNN_QUERY_INPUT_ATTR	√	√	√
3	RKNN_QUERY_OUTPUT_ATTR	√	√	√
4	RKNN_QUERY_PERF_DETAIL	√	√	×
5	RKNN_QUERY_PERF_RUN	√	√	×
6	RKNN_QUERY_SDK_VERSION	√	√	√
7	RKNN_QUERY_MEM_SIZE	√	√	√
8	RKNN_QUERY_CUSTOM_STRING	√	√	√
9	RKNN_QUERY_NATIVE_INPUT_ATTR	√	√	√
10	RKNN_QUERY_NATIVE_OUTPUT_ATTR	√	√	√
11	RKNN_QUERY_NATIVE_NC1HWC2_INPUT_ATTR	√	√	√
12	RKNN_QUERY_NATIVE_NC1HWC2_OUTPUT_ATTR	√	√	√
13	RKNN_QUERY_NATIVE_NHWC_INPUT_ATTR	√	√	√
14	RKNN_QUERY_NATIVE_NHWC_OUTPUT_ATTR	√	√	√

7.2 API 函数介绍

7.2.1 rknn_init

rknn_init 初始化函数将创建 rknn_context 对象、加载 RKNN 模型以及根据 flag 和 rknn_init_extend 结构体执行特定的初始化行为。

API	rknn_init
功能	初始化 rknn，创建 rknn_context 对象、加载 RKNN 模型。
参数	rknn_context *context：rknn_context 指针。函数调用之后，context 将会被赋值。
	void *model：RKNN 模型的二进制数据或者 RKNN 模型路径。
	uint32_t size：当 model 是二进制数据，表示模型大小，当 model 是路径，则设置为 0。
	uint32_t flag：特定的初始化标志。目前，仅支持如下标志：RKNN_FLAG_COLLECT_PERF_MASK：用于运行时查询网络各层时间。RKNN_FLAG_MEM_ALLOC_OUTSIDE：用于表示模型输入、输出、权重、中间 tensor 内存全部由用户分配。 RKNN_FLAG_SHARE_WEIGHT_MEM：用于表示共享另个模型的 weight 权重
	rknn_init_extend：特定初始化时的扩展信息。没有使用，传入 NULL 即可。如果需要共享模型 weight 内存，则需要传入另个模型 rknn_context 指针
返回值	int 错误码（见 rknn 返回值错误码）。

示例代码如下：

```
rknn_context ctx;
int ret = rknn_init(&ctx, model_data, model_data_size, 0, NULL);
```

7.2.2 rknn_destroy

rknn_destroy 函数将释放传入的 rknn_context 及其相关资源。

API	rknn_destroy
功能	销毁 rknn_context 对象及其相关资源。
参数	rknn_context context：要销毁的 rknn_context 对象。
返回值	int 错误码（见 rknn 返回值错误码）。

示例代码如下：

```
rknn_context ctx;
int ret = rknn_destroy (ctx);
```

7.2.3 rknn_query

rknn_query 函数能够查询获取到模型输入输出信息、逐层运行时间、模型推理的总时间、SDK 版本、内存占用信息、用户自定义字符串等信息。

API	rknn_query
功能	查询模型与 SDK 的相关信息。
参数	rknn_context context：rknn_context 对象。
	rknn_query_cmd ：查询命令。
	void* info：存放返回结果的结构体变量。
	uint32_t size：info 对应的结构体变量的大小。
返回值	int 错误码（见 rknn 返回值错误码）

当前 SDK 支持的查询命令如下表所示：

查询命令	返回结果结构体	功能
RKNN_QUERY_IN_OUT_NUM	rknn_input_output_num	查询输入输出 tensor 个数
RKNN_QUERY_INPUT_ATTR	rknn_tensor_attr	查询输入 tensor 属性
RKNN_QUERY_OUTPUT_ATTR	rknn_tensor_attr	查询输出 tensor 属性
RKNN_QUERY_PERF_DETAIL	rknn_perf_detail	查询网络各层运行时间，需要调用 rknn_init 接口时，设置 RKNN_FLAG_COLLECT_PERF_MASK 标志才能生效
RKNN_QUERY_PERF_RUN	rknn_perf_run	查询推理模型（不包含设置输入/输出）的耗时，单位是微秒
RKNN_QUERY_SDK_VERSION	rknn_sdk_version	查询 SDK 版本
RKNN_QUERY_MEM_SIZE	rknn_mem_size	查询分配给权重和网络中间 tensor 的内存大小
RKNN_QUERY_CUSTOM_STRING	rknn_custom_string	查询 RKNN 模型里面的用户自定义字符串信息
RKNN_QUERY_NATIVE_INPUT_ATTR	rknn_tensor_attr	使用零拷贝 API 接口时,查询原生输入 tensor 属性，它是 NPU 直接读取的模型输入属性
RKNN_QUERY_NATIVE_OUTPUT_ATTR	rknn_tensor_attr	使用零拷贝 API 接口时,查询原生输出 tensor 属性，它是 NPU 直接输出的模型输出属性
RKNN_QUERY_NATIVE_NC1HWC2_INPUT_ATTR	rknn_tensor_attr	使用零拷贝 API 接口时,查询原生输入 tensor 属性，它是 NPU 直接读取的模型输入属性与 RKNN_QUERY_NATIVE_INPUT_ATTR 查询结果一致
RKNN_QUERY_NATIVE_NC1HWC2_OUTPUT_ATTR	rknn_tensor_attr	使用零拷贝 API 接口时,查询原生输出 tensor 属性，它是 NPU 直接输出的模型输出属与 RKNN_QUERY_NATIVE_OUTPUT_ATTR 查询结果一致性

RKNN_QUERY_NATIVE_NHWC_INPUT_ATTR	rknn_tensor_attr	使用零拷贝 API 接口时,查询原生输入 tensor 属性与 RKNN_QUERY_NATIVE_INPUT_ATTR 查询结果一致
RKNN_QUERY_NATIVE_NHWC_OUTPUT_ATTR	rknn_tensor_attr	使用零拷贝 API 接口时,查询原生输出 NHWC tensor 属性

各个指令用法的详细说明，如下：

7.2.3.1 查询 SDK 版本

传入 RKNN_QUERY_SDK_VERSION 命令可以查询 RKNN SDK 的版本信息。其中需要先创建 rknn_sdk_version 结构体对象。

示例代码如下：

```
rknn_sdk_version version;
ret = rknn_query(ctx, RKNN_QUERY_SDK_VERSION, &version, sizeof(rknn_sdk_version));
printf("sdk api version: %s\n", version.api_version);
printf("driver version: %s\n", version.drv_version);
```

7.2.3.2 查询输入输出 tensor 个数

在 rknn_init 接口调用完毕后，传入 RKNN_QUERY_IN_OUT_NUM 命令可以查询模型输入输出 tensor 的个数。其中需要先创建 rknn_input_output_num 结构体对象。

示例代码如下：

```
rknn_input_output_num io_num;
ret = rknn_query(ctx, RKNN_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
printf("model input num: %d, output num: %d\n", io_num.n_input, io_num.n_output);
```

7.2.3.3 查询输入 tensor 属性(用于通用 API 接口)

在 rknn_init 接口调用完毕后，传入 RKNN_QUERY_INPUT_ATTR 命令可以查询模型输入 tensor 的属性。其中需要先创建 rknn_tensor_attr 结构体对象。(注意：RV1106/RV1103 查询出来的 tensor 是原始输入 native 的 tensor)

示例代码如下：

```
rknn_tensor_attr input_attrs[io_num.n_input];
memset(input_attrs, 0, sizeof(input_attrs));
for (int i = 0; i < io_num.n_input; i++)
```

```
    {
        input_attrs[i].index = i;
        ret = rknn_query(ctx, RKNN_QUERY_INPUT_ATTR, &(input_attrs[i]),
sizeof(rknn_tensor_attr));
    }
```

7.2.3.4 查询输出 tensor 属性(用于通用 API 接口)

在 rknn_init 接口调用完毕后，传入 RKNN_QUERY_OUTPUT_ATTR 命令可以查询模型输出 tensor 的属性。其中需要先创建 rknn_tensor_attr 结构体对象。

示例代码如下：

```
    rknn_tensor_attr output_attrs[io_num.n_output];
    memset(output_attrs, 0, sizeof(output_attrs));
    for (int i = 0; i < io_num.n_output; i++)
    {
        output_attrs[i].index = i;
        ret = rknn_query(ctx, RKNN_QUERY_OUTPUT_ATTR, &(output_attrs[i]),
sizeof(rknn_tensor_attr));
    }
```

7.2.3.5 查询模型推理的逐层耗时

在 rknn_run 接口调用完毕后，rknn_query 接口传入 RKNN_QUERY_PERF_DETAIL 可以查询网络推理时逐层的耗时，单位是微秒。使用该命令的前提是，在 rknn_init 接口的 flag 参数需要包含 RKNN_FLAG_COLLECT_PERF_MASK 标志。

示例代码如下：

```
    rknn_context ctx;
    int ret = rknn_init(&ctx, model_data, model_data_size, RKNN_FLAG_COLLECT_PERF_MASK, NULL);
    ...
    ret = rknn_run(ctx,NULL);
    ...
    rknn_perf_detail perf_detail;
    ret = rknn_query(ctx, RKNN_QUERY_PERF_DETAIL, &perf_detail, sizeof(perf_detail));
```

7.2.3.6 查询模型推理的总耗时

在 rknn_run 接口调用完毕后，rknn_query 接口传入 RKNN_QUERY_PERF_RUN 可以查询上模型推理（不包含设置输入/输出）的耗时，单位是微秒。示例代码如下：

```
rknn_context ctx;
int ret = rknn_init(&ctx, model_data, model_data_size, 0, NULL);
...
ret = rknn_run(ctx,NULL);
...
rknn_perf_run perf_run;
ret = rknn_query(ctx, RKNN_QUERY_PERF_RUN, &perf_run, sizeof(perf_run));
```

7.2.3.7 查询模型的内存占用情况

在 rknn_init 接口调用完毕后，当用户需要自行分配网络的内存时，rknn_query 接口传入 RKNN_QUERY_MEM_SIZE 可以查询模型的权重、网络中间 tensor 的内存（不包括输入和输出）、推演模型所用的所有 DMA 内存的以及 SRAM 内存（如果 sram 没开或者没有此项功能则为 0）的占用情况。使用该命令的前提是在 rknn_init 接口的 flag 参数需要包含 RKNN_FLAG_MEM_ALLOC_OUTSIDE 标志。

示例代码如下：

```
rknn_context ctx;
int ret = rknn_init(&ctx, model_data, model_data_size, RKNN_FLAG_MEM_ALLOC_OUTSIDE , NULL);
rknn_mem_size mem_size;
ret = rknn_query(ctx, RKNN_QUERY_MEM_SIZE, &mem_size, sizeof(mem_size));
```

7.2.3.8 查询模型里用户自定义字符串

在 rknn_init 接口调用完毕后，当用户需要查询生成 RKNN 模型时加入的自定义字符串，rknn_query 接口传入 RKNN_QUERY_CUSTOM_STRING 可以获取该字符串。

示例代码如下：

```
rknn_context ctx;
int ret = rknn_init(&ctx, model_data, model_data_size, 0, NULL);
rknn_custom_string custom_string;
ret = rknn_query(ctx, RKNN_QUERY_CUSTOM_STRING, &custom_string, sizeof(custom_string));
```

7.2.3.9 查询原始输入 tensor 属性(用于零拷贝 API 接口)

在 rknn_init 接口调用完毕后，传入 RKNN_QUERY_NATIVE_INPUT_ATTR 命令可以查询模型原生输入 tensor 的属性。其中需要先创建 rknn_tensor_attr 结构体对象。

示例代码如下：

```
    rknn_tensor_attr input_attrs[io_num.n_input];
    memset(input_attrs, 0, sizeof(input_attrs));
    for (int i = 0; i < io_num.n_input; i++)
    {
        input_attrs[i].index = i;
        ret = rknn_query(ctx, RKNN_QUERY_NATIVE_INPUT_ATTR, &(input_attrs[i]),
sizeof(rknn_tensor_attr));
    }
```

7.2.3.10 查询原始输出 tensor 属性(用于零拷贝 API 接口)

在 rknn_init 接口调用完毕后，传入 RKNN_QUERY_NATIVE_OUTPUT_ATTR 命令可以查询模型原生输出 tensor 的属性。其中需要先创建 rknn_tensor_attr 结构体对象。

示例代码如下：

```
    rknn_tensor_attr output_attrs[io_num.n_output];
    memset(output_attrs, 0, sizeof(output_attrs));
    for (int i = 0; i < io_num.n_output; i++)
    {
        output_attrs[i].index = i;
        ret = rknn_query(ctx, RKNN_QUERY_NATIVE_OUTPUT_ATTR, &(output_attrs[i]),
sizeof(rknn_tensor_attr));
    }
```

7.2.3.11 查询原始输入 tensor 属性(用于零拷贝 API 接口)

在 rknn_init 接口调用完毕后，传入 RKNN_QUERY_NATIVE_NHWC_INPUT_ATTR 命令可以查询模型 NHWC 格式输入 tensor 的属性。其中需要先创建 rknn_tensor_attr 结构体对象。

示例代码如下：

```
    rknn_tensor_attr input_attrs[io_num.n_input];
    memset(input_attrs, 0, sizeof(input_attrs));
```

```
    for (int i = 0; i < io_num.n_input; i++)
    {
        input_attrs[i].index = i;
        ret = rknn_query(ctx, RKNN_QUERY_NATIVE_NHWC_INPUT_ATTR, &(input_attrs[i]),
sizeof(rknn_tensor_attr));
    }
```

7.2.3.12 查询原始输出 tensor 属性(用于零拷贝 API 接口)

在 rknn_init 接口调用完毕后，传入 RKNN_QUERY_NATIVE_NHWC_OUTPUT_ATTR 命令可以查询模型 NHWC 格式输出 tensor 的属性。其中需要先创建 rknn_tensor_attr 结构体对象。

示例代码如下：

```
    rknn_tensor_attr output_attrs[io_num.n_output];
    memset(output_attrs, 0, sizeof(output_attrs));
    for (int i = 0; i < io_num.n_output; i++)
    {
        output_attrs[i].index = i;
        ret = rknn_query(ctx, RKNN_QUERY_NATIVE_NHWC_OUTPUT_ATTR, &(output_attrs[i]),
sizeof(rknn_tensor_attr));
    }
```

7.2.4 rknn_inputs_set

通过 rknn_inputs_set 函数可以设置模型的输入数据。该函数能够支持多个输入，其中每个输入是 rknn_input 结构体对象，在传入之前用户需要设置该对象。（注：RV1106/RV1103 不支持这个接口）

API	rknn_inputs_set
功能	设置模型输入数据。
参数	rknn_context context：rknn_context 对象。
	uint32_t n_inputs：输入数据个数。
	rknn_input inputs[]：输入数据数组，数组每个元素是 rknn_input 结构体对象。
返回值	int 错误码（见 rknn 返回值错误码）

示例代码如下:

```
    rknn_input inputs[1];
    memset(inputs, 0, sizeof(inputs));
```

```
    inputs[0].index = 0;
    inputs[0].type = RKNN_TENSOR_UINT8;
    inputs[0].size = img_width*img_height*img_channels;
    inputs[0].fmt = RKNN_TENSOR_NHWC;
    inputs[0].buf = in_data;
    inputs[0].pass_through = 0;
    ret = rknn_inputs_set(ctx, 1, inputs);
```

7.2.5 rknn_run

rknn_run 函数将执行一次模型推理，调用之前需要先通过 rknn_inputs_set 函数或者零拷贝的接口设置输入数据。

API	rknn_run
功能	执行一次模型推理。
参数	rknn_context context：rknn_context 对象。
	rknn_run_extend* extend：保留扩展，当前没有使用，传入 NULL 即可。
返回值	int 错误码（见 rknn 返回值错误码）

示例代码如下：

```
    ret = rknn_run(ctx, NULL);
```

7.2.6 rknn_outputs_get

rknn_outputs_get 函数可以获取模型推理的输出数据。该函数能够一次获取多个输出数据。其中每个输出是 rknn_output 结构体对象，在函数调用之前需要依次创建并设置每个 rknn_output 对象。

对于输出数据的 buffer 存放可以采用两种方式：一种是用户自行申请和释放，此时 rknn_output 对象的 is_prealloc 需要设置为 1，并且将 buf 指针指向用户申请的 buffer；另一种是由 rknn 来进行分配，此时 rknn_output 对象的 is_prealloc 设置为 0 即可，函数执行之后 buf 将指向输出数据。（注：RV1106/RV1103 不支持这个接口）

API	rknn_outputs_get
功能	获取模型推理输出。
参数	rknn_context context：rknn_context 对象。
	uint32_t n_outputs：输出数据个数。
	rknn_output outputs[]：输出数据的数组，其中数组每个元素为 rknn_output 结构体对象，代表模型的一个输出。

	rknn_output_extend* extend：保留扩展，当前没有使用，传入 NULL 即可
返回值	int 错误码（见 rknn 返回值错误码）

示例代码如下：

```
rknn_output outputs[io_num.n_output];
memset(outputs, 0, sizeof(outputs));
for (int i = 0; i < io_num.n_output; i++)
{
    outputs[i].index = i;
    outputs[i].is_prealloc = 0;
    outputs[i].want_float = 1;
}
ret = rknn_outputs_get(ctx, io_num.n_output, outputs, NULL);
```

7.2.7 rknn_outputs_release

rknn_outputs_release 函数将释放 rknn_outputs_get 函数得到的输出的相关资源。

API	rknn_outputs_release
功能	释放 rknn_output 对象。
参数	rknn_context context：rknn_context 对象。
	uint32_t n_outputs：输出数据个数。
	rknn_output outputs[]：要销毁的 rknn_output 数组。
返回值	int 错误码（见 rknn 返回值错误码）

示例代码如下所示：

```
ret = rknn_outputs_release(ctx, io_num.n_output, outputs);
```

7.3 数据结构定义

7.3.1 rknn_sdk_version

结构体 rknn_sdk_version 用来表示 RKNN SDK 的版本信息，结构体的定义如下：

成员变量	数据类型	含义
api_version	char[]	SDK 的版本信息。
drv_version	char[]	SDK 所基于的驱动版本信息。

7.3.2 rknn_input_output_num

结构体 rknn_input_output_num 表示输入输出 tensor 个数， 其结构体成员变量如下表所示:

成员变量	数据类型	含义
n_input	uint32_t	输入 tensor 个数
n_output	uint32_t	输出 tensor 个数

7.3.3 rknn_tensor_attr

结构体 rknn_tensor_attr 表示模型的 tensor 的属性，结构体的定义如下表所示：

成员变量	数据类型	含义
index	uint32_t	表示输入输出 tensor 的索引位置。
n_dims	uint32_t	Tensor 维度个数。
dims	uint32_t[]	Tensor 各维度值。
name	char[]	Tensor 名称。
n_elems	uint32_t	Tensor 数据元素个数。
size	uint32_t	Tensor 数据所占内存大小。
fmt	rknn_tensor_format	Tensor 维度的格式，有以下格式： RKNN_TENSOR_NCHW RKNN_TENSOR_NHWC RKNN_TENSOR_NC1HWC2
type	rknn_tensor_type	Tensor 数据类型，有以下数据类型： RKNN_TENSOR_FLOAT32 RKNN_TENSOR_FLOAT16 RKNN_TENSOR_INT8 RKNN_TENSOR_UINT8 RKNN_TENSOR_INT16 RKNN_TENSOR_UINT16 RKNN_TENSOR_INT32 RKNN_TENSOR_INT64 RKNN_TENSOR_BOOL

qnt_type	rknn_tensor_qnt_type	Tensor 量化类型，有以下的量化类型： RKNN_TENSOR_QNT_NONE：未量化； RKNN_TENSOR_QNT_DFP：动态定点量化； RKNN_TENSOR_QNT_AFFINE_ASYMMETRIC：非对称量化。
fl	int8_t	RKNN_TENSOR_QNT_DFP 量化类型的参数。
scale	float	RKNN_TENSOR_QNT_AFFINE_ASYMMETRIC 量化类型的参数。
w_stride	uint32_t	实际存储一行图像数据的像素数目，等于一行的有效数据像素数目 + 为硬件快速跨越到下一行而补齐的一些无效像素数目，单位是像素。
size_with_stride	uint32_t	实际存储图像数据所占的存储空间的大小（包括了补齐的无效像素的存储空间大小）。
pass_through	uint8_t	0 表示未转换的数据，1 表示转换后的数据，转换包括归一化和量化。
h_stride	uint32_t	仅用于多 batch 输入场景，且值由用户设置。目的是 NPU 正确地读取每 batch 数据的起始地址，它等于原始模型的输入高度+跨越下一列而补齐的无效像素数目。如果设置成 0，表示与原始模型输入高度一致。单位是像素。

7.3.4 rknn_perf_detail

结构体 rknn_perf_detail 表示模型的性能详情，结构体的定义如下表所示：（RV1106/RV1103 暂不支持）

成员变量	数据类型	含义
perf_data	char*	性能详情包含网络每层运行时间，能够直接打印出来查看。
data_len	uint64_t	存放性能详情的字符串数组的长度。

7.3.5 rknn_perf_run

结构体 rknn_perf_run 表示模型的总体性能，结构体的定义如下表所示：（RV1106/RV1103 暂不支持）

成员变量	数据类型	含义

run_duration	int64_t	网络总体运行（不包含设置输入/输出）时间，单位是微秒。

7.2.6 rknn_mem_size

结构体 rknn_mem_size 表示初始化模型时的内存分配情况，结构体的定义如下表所示：

成员变量	数据类型	含义
total_weight_size	uint32_t	模型的权重占用的内存大小。
total_internal_size	uint32_t	模型的中间 tensor 占用的内存大小。
total_dma_allocated_size	uint64_t	模型申请的所有 dma 内存之和。
total_sram_size	uint32_t	只针对 RK3588 有效，为 NPU 预留的系统 SRAM 大小。
free_sram_size	uint32_t	只针对 RK3588 有效，当前可用的空闲 SRAM 大小。
reserved[12]	uint32_t	预留

7.3.7 rknn_tensor_mem

结构体 rknn_tensor_mem 表示 tensor 的内存信息。结构体的定义如下表所示：

成员变量	数据类型	含义
virt_addr	void*	该 tensor 的虚拟地址。
phys_addr	uint64_t	该 tensor 的物理地址。
fd	int32_t	该 tensor 的文件描述符。
offset	int32_t	相较于文件描述符和虚拟地址的偏移量。
size	uint32_t	该 tensor 占用的内存大小。
flags	uint32_t	rknn_tensor_mem 的标志位，有以下标志： (1)RKNN_TENSOR_MEMORY_FALGS_ALLOC_INSIDE: 表明 rknn_tensor_mem 结构体由运行时创建。 (2)RKNN_TENSOR_MEMORY_FLAGS_FROM_FD: 表明 rknn_tensor_mem 结构体由 fd 构造。 (3)RKNN_TENSOR_MEMORY_FLAGS_FROM_PHYS: 表明 rknn_tensor_mem 结构体由物理地址构造。用户不用关注该标志。
priv_data	void*	内存的私有数据。

7.3.8 rknn_input

结构体 rknn_input 表示模型的一个数据输入，用来作为参数传入给 rknn_inputs_set 函数。结构体的定义如下表所示：

成员变量	数据类型	含义
index	uint32_t	该输入的索引位置。
buf	void*	输入数据的指针。
size	uint32_t	输入数据所占内存大小。
pass_through	uint8_t	设置为 1 时会将 buf 存放的输入数据直接设置给模型的输入节点，不做任何预处理。
type	rknn_tensor_type	输入数据的类型。
fmt	rknn_tensor_format	输入数据的格式。

7.3.9 rknn_output

结构体 rknn_output 表示模型的一个数据输出，用来作为参数传入给 rknn_outputs_get 函数，在函数执行后，结构体对象将会被赋值。结构体的定义如下表所示：

成员变量	数据类型	含义
want_float	uint8_t	标识是否需要将输出数据转为 float 类型输出
is_prealloc	uint8_t	标识存放输出数据是否是预分配。
index	uint32_t	该输出的索引位置。
buf	void*	输出数据的指针。
size	uint32_t	输出数据所占内存大小。

7.4 RKNN 返回值错误码

RKNN API 函数的返回值错误码定义如下表所示：

错误码	错误详情
RKNN_SUCC（0）	执行成功
RKNN_ERR_FAIL（-1）	执行出错
RKNN_ERR_TIMEOUT（-2）	执行超时
RKNN_ERR_DEVICE_UNAVAILABLE（-3）	NPU 设备不可用
RKNN_ERR_MALLOC_FAIL（-4）	内存分配失败
RKNN_ERR_PARAM_INVALID（-5）	传入参数错误
RKNN_ERR_MODEL_INVALID（-6）	传入的 RKNN 模型无效

RKNN_ERR_CTX_INVALID（-7）	传入的 rknn_context 无效
RKNN_ERR_INPUT_INVALID（-8）	传入的 rknn_input 对象无效
RKNN_ERR_OUTPUT_INVALID（-9）	传入的 rknn_output 对象无效
RKNN_ERR_DEVICE_UNMATCH（-10）	版本不匹配
RKNN_ERR_INCOMPATILE_OPTIMIZATION_LEVEL_VERSION（-12）	RKNN 模型设置了优化等级的选项，但是和当前驱动不兼容
RKNN_ERR_TARGET_PLATFORM_UNMATCH（-13）	RKNN 模型和当前平台不兼容